Open the Modules

Déclarer le plan de navigation et remplacer la fausse barre (`useState`) par un vrai routeur.

---

*Durée indicative : **35 à 40 min** · Organisation : **individuel** · Évaluation :** non noté*

Support : starter `jour-04-bis/tp-01-open-the-modules`, dans l'IDE

Contexte

La flotte du jour 2 est typée, découpée, filtrable et responsive. Tout vit encore **sur un seul écran**. Le starter simule les modules avec quatre boutons et `useState("fleet")` : l'URL ne change pas, F5 ramène toujours à la flotte, le bouton précédent du navigateur ne sert à rien.

Aujourd'hui Orbital Command devient une application à plusieurs destinations. Chaque écran important a une URL : on peut la rafraîchir, la partager, y revenir.

Une SPA se navigue avec des **liens**, pas des boutons qui changent un état local. `NavLink` sait si la destination est active. La barre est partagée : elle ne se recopie pas dans chaque page.

Les destinations métier :

- `/fleet` — registre de flotte
- `/missions` — liste des missions
- `/control` — centre de contrôle
- `/alerts` — journal des alertes
- `/fleet/:vehicleId` et `/missions/:missionId` — fiches (coquilles pour ce TP)
- `*` — aucune route ne correspond

`:vehicleId` est un **segment variable**. `/fleet/unknown` (route OK, ressource non) n'est **pas** `/pizza-on-mars` (aucune route). Le second se règle avec `path="*"`. Le premier attend le TP 2.

Résultat attendu

1. `npm run dev` tourne.

2. Cliquer Flotte / Missions / Contrôle / Alertes change **l'URL et** le contenu.

3. Le lien actif est visible (libellé ou `aria-current`, pas seulement une couleur).

4. Précédent / suivant du navigateur fonctionnent. F5 sur `/control` laisse l'utilisateur sur Contrôle, barre comprise.

5. Taper `/fleet/aquila-1` et `/missions/odyssey-01` affiche une page de détail (même vide, titre clair). `/pizza-on-mars` affiche une 404, pas un crash.

6. `FleetPage` affiche la vraie flotte. Aucune page ne duplique la barre. Plus aucun `useState` pour choisir le module.

7. `npx tsc --noEmit` est vert.

Étapes

Travaillez seul, dans le starter. Ordre imposé. Ne passez à la suivante que si la précédente est visible (navigateur ou type-check).

1. Vérifier React Router

La dépendance `react-router-dom` est déjà dans `package.json`. Installez puis lancez :

```
npm install
npm run dev
```

Si elle manquait : `npm install react-router-dom`. Dans `main.tsx`, enveloppez `<App />` avec `BrowserRouter`. Sans ça, les hooks de navigation planteront.

2. Déclarer les chemins au même endroit

Créez `src/routing/paths.ts`. Exportez des constantes, pas des chaînes recopiées :

- `FLEET` → `/fleet`
- `MISSIONS` → `/missions`
- `CONTROL` → `/control`
- `ALERTS` → `/alerts`
- `vehiclePath(id: string)` → `/fleet/${id}`
- `missionPath(id: string)` → `/missions/${id}`

Chemins métier, anglais, minuscules, qui commencent par `/`. Pas de `/page1`.

3. Pages manquantes + Routes

Les quatre modules existent déjà (`FleetPage`, `MissionsPage`, `ControlPage`, `AlertsPage`). Ajoutez si besoin :

- `src/pages/VehicleDetailPage.tsx`
- `src/pages/MissionDetailPage.tsx`
- `src/pages/NotFoundPage.tsx`

Pour ce TP, un `<h1>` unique suffit sur les fiches et la 404.

Dans `App.tsx`, déclarez `Routes` / `Route` avec les constantes de `paths.ts`. Reliez les quatre modules, les deux routes dynamiques, et `path="*"` vers `NotFoundPage`. Supprimez le `useState` de module.

4. Extraire un layout

Créez `src/components/AppLayout.tsx` :

- quatre `NavLink` (plus un lien Briefing vers `/` si le starter a `HomePage`) ;
- sous la barre, `{children}` **ou** `<Outlet />` ;
- aucun texte métier des pages (« Aquila », liste des missions) dans ce fichier.

Route parente `element={<AppLayout />}` + routes enfants. La 404 peut rester **dans** le layout (barre visible) ou hors — soyez cohérent. Ne recopiez pas `AppLayout` dans chaque page.

Sur chaque `NavLink` : `className={({ isActive }) => ...}` **et** un indice non coloré (`aria-current="page"`, suffixe `·`). Tab jusqu'au lien, Enter, focus visible.

5. Vérifier

- Cliquer Flotte → Missions → Contrôle → Alertes.
- Bouton précédent : revenir à Contrôle.
- F5 sur `/alerts` et `/control`.
- Saisir `/fleet/aquila-1`, `/missions/odyssey-01`, `/pizza-on-mars`.
- Inspecter : **une seule** barre dans le DOM.

Incident à refuser : quatre boutons + `const [module, setModule] = useState("fleet")`.

```
npx tsc --noEmit
```

Livrables

- `src/routing/paths.ts`
- `AppLayout` + `BrowserRouter` + `Routes`
- fiches et 404 branchées (même vides)
- type-check sans erreur
- commit :

```
git add src package.json package-lock.json
git commit -m "feat(session-04): persistent module navigation"
```

Montrez le navigateur au formateur (changement d'URL → changement d'écran, F5 conservé) avant le TP 2.

Suite — TP 2 Resource Detail

Les modules s'ouvrent, mais on ne peut pas encore **ouvrir un véhicule ou une mission**. Au TP 2 vous lirez `:vehicleId` / `:missionId` avec `useParams`, vous traiterez la ressource absente, et vous habillerez la 404 — sans confondre `/fleet/unknown` et `/pizza-on-mars`.